



Capítulo 1

Objetivos:

Explicar la diferencia entre automatización de pruebas y RPA

Conocer la arquitectura de Robot Framework y sus librerías nativas

Instalar el entorno: Python, Robot Framework, VS Code

Escribir y ejecutar tu primera suite .robot

Fundamentos de Automatización y Robot Framework

1.1: Automatización de Pruebas vs. RPA: Diferencias, Casos de Uso y Alcance

- Introducción

Dos términos se confunden mucho: automatización de pruebas y RPA.

Ambos usan software para ejecutar tareas automáticamente, pero tienen objetivos y audiencias distintas. Vamos a aclararlo.

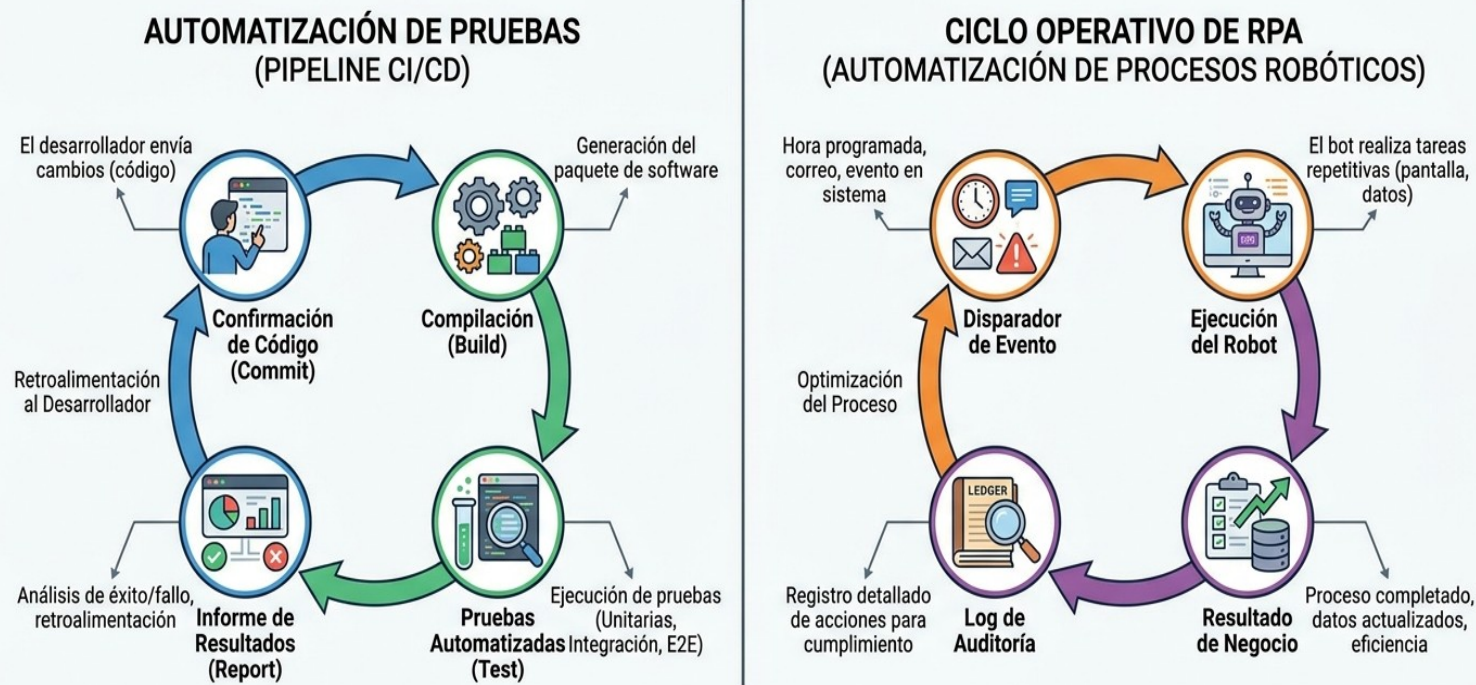
¿Qué es la Automatización de Pruebas?

- ▶ Verifica que un sistema de software funciona correctamente.
- ▶ Reemplaza o complementa pruebas manuales de QA.
- ▶ Produce reportes de pass/fail para el equipo de desarrollo.
- ▶ Se integra en pipelines de CI/CD con cada cambio de código.



¿Qué es la RPA?

COMPARACIÓN: CICLO DE VIDA DE AUTOMATIZACIÓN DE PRUEBAS (CI/CD) VS. RPA



ENFOQUE: Calidad del Código vs. Eficiencia Operativa
OBJETIVO: Detección de Errores vs. Automatización de Tareas Repetitivas

- ▶ Ejecuta procesos de negocio de extremo a extremo de forma autónoma.
- ▶ No valida: produce resultados operativos (archivos, registros, correos).
- ▶ Actúa como un empleado realizando tareas repetitivas en pantalla.
- ▶ Se dispara por horario o por eventos (correo, archivo, etc.).

Analogía: Inspector vs. Operario

- ▶ Automatización de pruebas → un inspector de calidad que revisa cada pieza.
- ▶ RPA → un operario que ensambla esas piezas en la línea de producción.
- ▶ Ambos trabajan en la misma fábrica, con roles distintos.
- ▶ Confundirlos te hace elegir la herramienta equivocada para el problema.

Comparación Técnica: Pruebas vs. RPA

Dimensión	Automatización de Pruebas	RPA
Objetivo principal	Verificar calidad del software	Ejecutar procesos de negocio
Actor beneficiado	Equipo de QA / Desarrollo	Áreas operativas / Negocio
Entrada típica	Casos de prueba, requerimientos	Datos operativos, reglas de negocio
Salida esperada	Reporte de pass/fail	Resultado del proceso (archivo, registro, correo)
Ciclo de vida	Ligado al ciclo de desarrollo (CI/CD)	Ligado al ciclo operativo del negocio
Tolerancia a cambios de UI	Media (se actualiza con el producto)	Baja (los sistemas legacy cambian poco)
Manejo de errores	Falla y reporta	Reintenta, escala o registra excepción
Herramientas comunes	Robot Framework, Selenium, Cypress	Robot Framework, UiPath, Power Automate

Cómo Elegir el Enfoque Correcto

- ▶ Pregunta clave: ¿necesitas verificar un sistema o ejecutar un proceso?
- ▶ Si el resultado es un reporte de calidad → automatización de pruebas.
- ▶ Si el resultado es un artefacto operativo (registro, archivo, correo) → RPA.
- ▶ El ciclo de vida determina la integración: CI/CD vs. ciclo operativo del negocio.
- ▶ El manejo de errores también diferencia: reportar fallos vs. reintentar y escalar.
- ▶ Robot Framework cubre ambos dominios; la arquitectura modular lo hace posible.

1.2: Ecosistema Robot

Framework: arquitectura, librerías nativas y alcance de RFCP

- Introducción

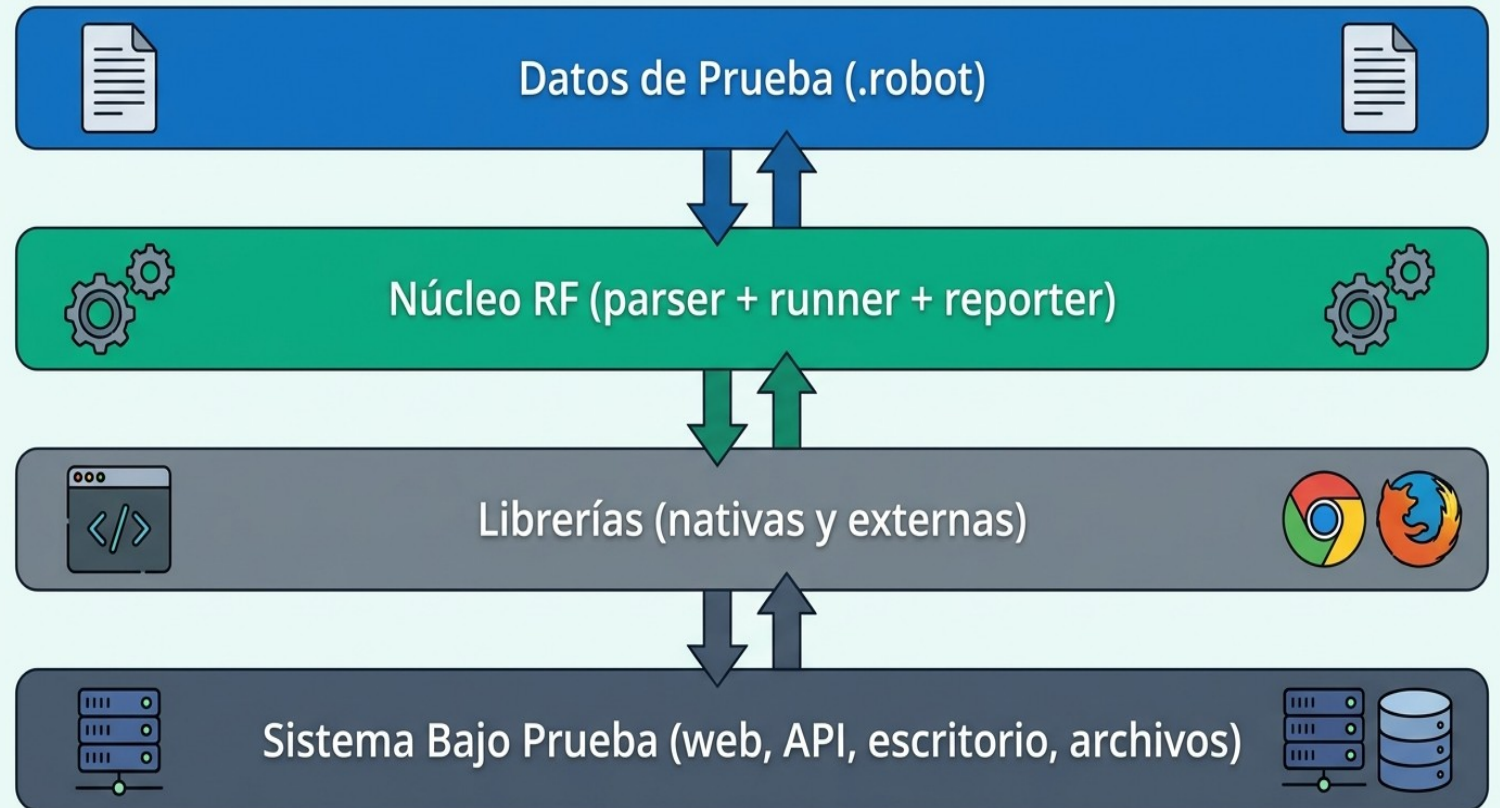
Robot Framework no es solo "una herramienta de pruebas".

Es un ecosistema: un núcleo extensible + librerías nativas +
librerías de la comunidad (Selenium, Requests, etc.).

Arquitectura en Capas de Robot Framework

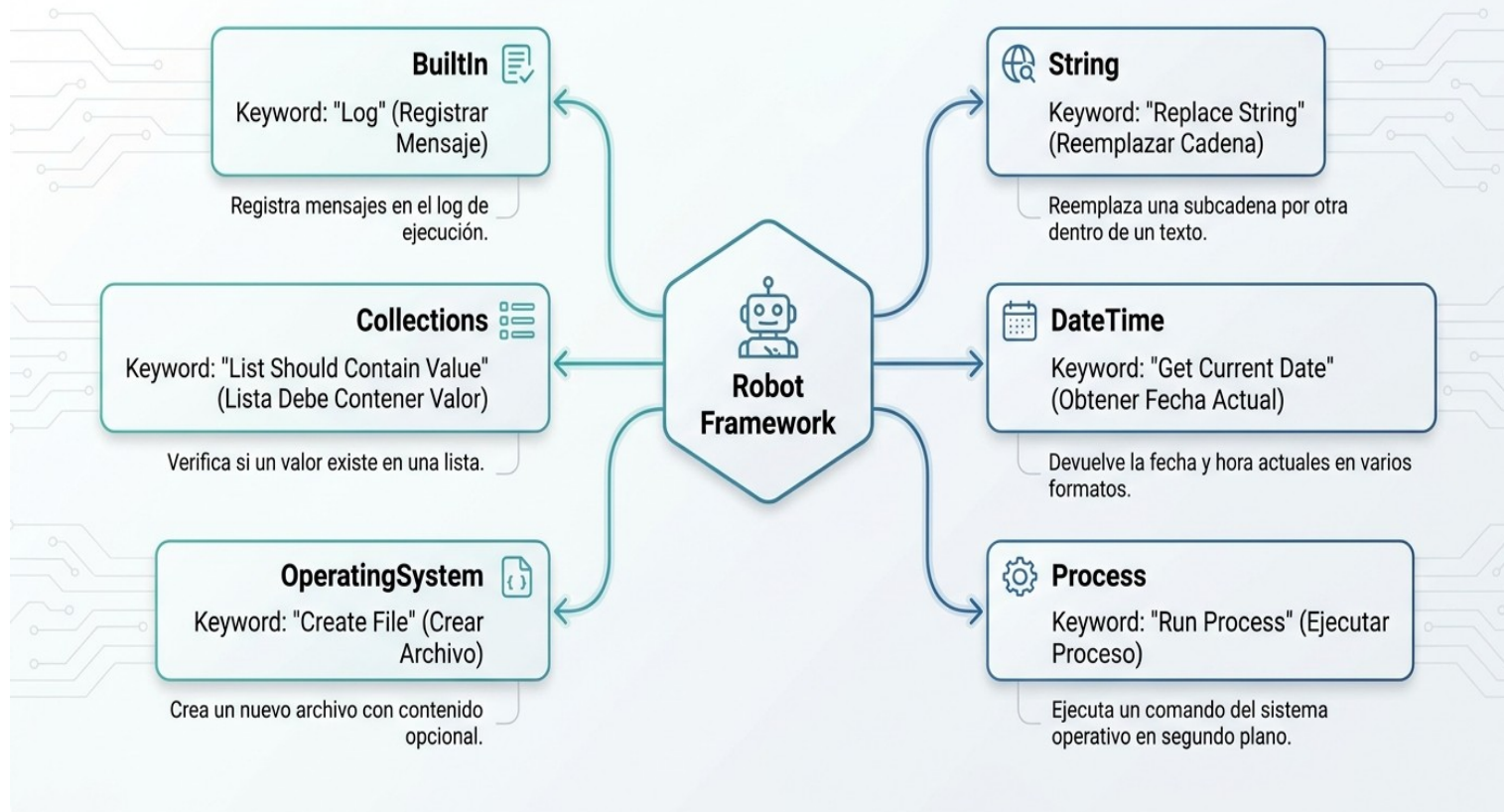
- Diseñado con separación de responsabilidades, no es un monolito cerrado.
- Capa 1: archivos .robot / .resource escritos por el equipo.
- Capa 2: núcleo que parsea, ejecuta y reporta.
- Capa 3: librerías que exponen keywords al motor.

DIAGRAMA DE CAPAS DE ROBOT FRAMEWORK



Librerías Nativas de Robot Framework

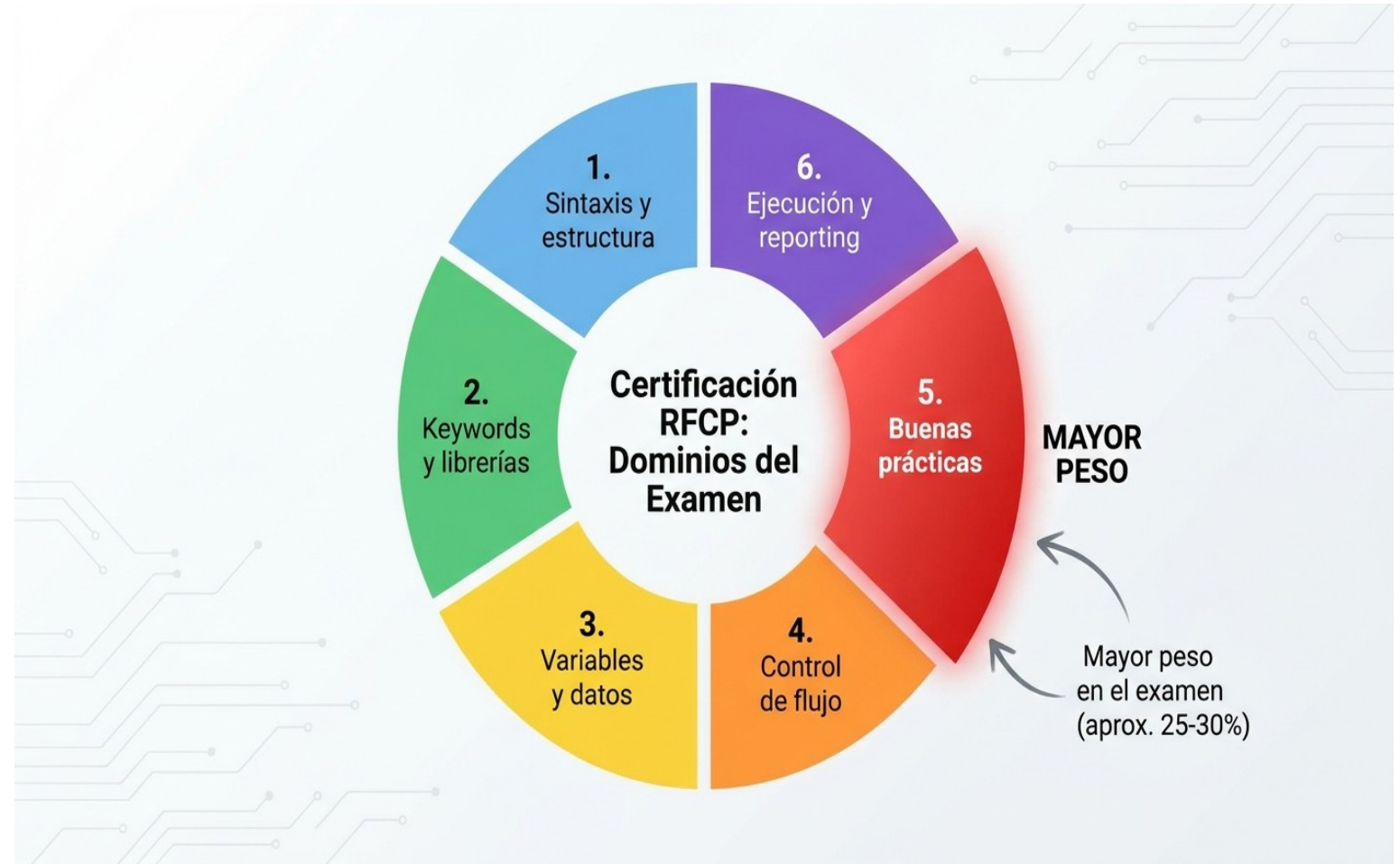
Mapa Conceptual: Librerías Nativas de Robot Framework y sus Keywords



- ▶ Se instalan automáticamente con Robot Framework, sin pip install extra.
- ▶ Representan el vocabulario base de cualquier suite de automatización.
- ▶ BuiltIn se importa automáticamente; las demás requieren declaración.
- ▶ Cubren operaciones fundamentales: datos, archivos, texto, fechas y procesos.

Certificación RFCP: ¿Qué Evalúa?

- ▶ Certificación oficial emitida por la Robot Framework Foundation.
- ▶ Evalúa dominio del lenguaje y la plataforma central, no librerías de dominio.
- ▶ NO incluye SeleniumLibrary, RequestsLibrary ni herramientas específicas.
- ▶ Valida competencia objetiva ante organizaciones contratantes.



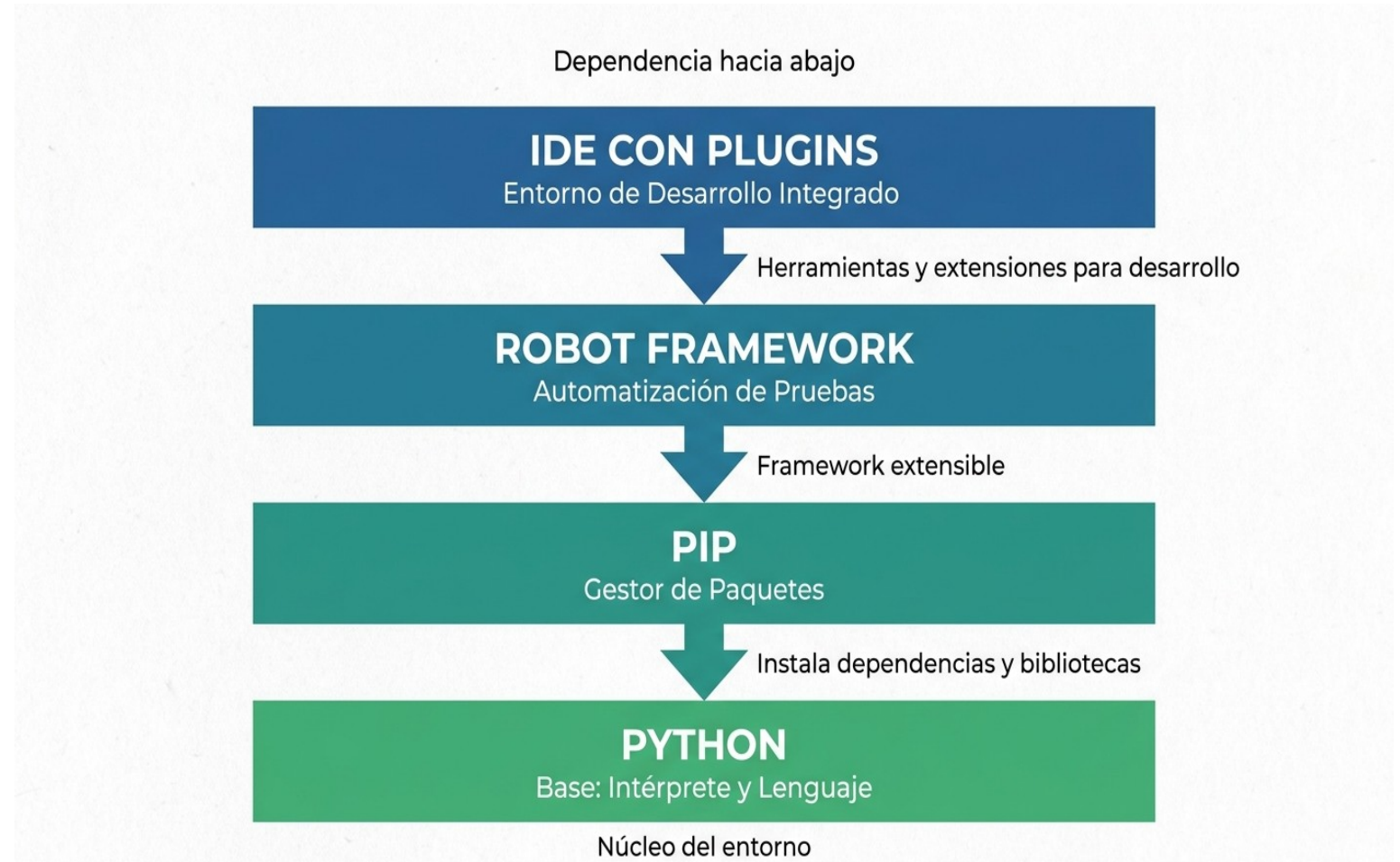
1.3: Instalación y configuración del entorno: Python, RF, IDE y plugins

- Introducción

Un entorno mal instalado es la causa más común de errores confusos al empezar. Por eso instalamos en orden: Python → pip → Robot Framework → IDE, y verificamos cada paso antes de avanzar.

Arquitectura del Entorno de Trabajo

- ▶ Las 4 capas deben instalarse en orden estricto.
- ▶ Python es la base de todo el ecosistema.
- ▶ pip gestiona la instalación de RF y librerías.
- ▶ El IDE potencia la productividad del desarrollador.



Verificación Integral del Entorno

```
# 1. Verificar Python
```

```
python --version
```

```
# 2. Verificar pip y listar paquetes instalados relacionados con RF
```

```
pip list | grep -i robot
```

```
# 3. Verificar el motor de Robot Framework
```

```
robot --version
```

```
# 4. Verificar una librería específica (ejemplo: SeleniumLibrary)
```

```
python -c "import SeleniumLibrary; print('SeleniumLibrary OK')"
```

1.4: Estructura de un Test Case: Secciones, Keywords Integradas y Primer Suite

- Introducción

Al escribir tu primera prueba, el reto no es la lógica: es saber dónde va cada cosa. Un archivo .robot tiene 4 secciones fijas, cada una con una responsabilidad clara.

Las Cuatro Secciones de un Archivo .robot

Sección	Encabezado	Propósito principal
Settings	Settings	Importar librerías, recursos y configurar Setup/Teardown globales
Variables	Variables	Declarar variables reutilizables a nivel de suite
Keywords	Keywords	Definir keywords propias (bloques reutilizables de pasos)
Test Cases	Test Cases	Declarar los casos de prueba individuales

Keywords Esenciales de la Librería BuiltIn

Categoría	Keyword	Descripción
Logging	Log	Imprime un mensaje en el log de ejecución
Logging	Log Many	Imprime múltiples valores de una vez
Aserciones	Should Be Equal	Falla si dos valores no son iguales
Aserciones	Should Be Equal As Integers	Compara valores como enteros
Aserciones	Should Contain	Falla si un string no contiene otro
Aserciones	Should Be True	Evalúa una expresión Python como verdadera
Variables	Set Variable	Asigna un valor a una variable en tiempo de ejecución
Variables	Set Suite Variable	Hace visible una variable en todo el suite
Control	Sleep	Pausa la ejecución un número de segundos
Control	Fail	Fuerza el fallo del test con un mensaje personalizado

Resumen del capítulo

- La **automatización de pruebas** tiene como objetivo verificar la calidad del software; su producto es un reporte de resultados (pass/fail).
- La **RPA** tiene como objetivo ejecutar procesos de negocio de forma autónoma; su producto es el resultado operativo del proceso.
- Ambas disciplinas pueden implementarse con **Robot Framework**, lo que convierte a esta herramienta en una opción estratégicamente versátil.
- La elección entre automatización de pruebas y RPA no depende de la herramienta, sino del **problema a resolver**: ¿necesitas verificar un sistema o ejecutar un proceso?
- El manejo de errores, el ciclo de vida y la audiencia beneficiada son los tres ejes que mejor diferencian ambos enfoques en la práctica.



Resumen del capítulo

- Robot Framework se organiza en cuatro capas: datos de prueba, núcleo, librerías y sistema bajo prueba; cada capa tiene una responsabilidad única y bien delimitada.
- El núcleo está formado por tres subsistemas: parser, runner y reporter; no contiene lógica de dominio, que reside exclusivamente en las librerías.
- Las librerías nativas (BuiltIn, Collections, OperatingSystem, String, DateTime, Proces) están disponibles desde la instalación base y cubren operaciones fundamentales de cualquier suite.
- BuiltIn es la única librería que se importa automáticamente; las demás deben declararse en **Settings** .
- La certificación RFCP evalúa el dominio del lenguaje y la plataforma central, no librerías de dominio específico; su alcance cubre sintaxis, keywords, variables, control de flujo, ejecución y reporting.



Práctica 1: Instalación del entorno y ejecución del primer test case

Objetivo:

Configurar Python, Robot Framework y VS Code; escribir y ejecutar un test case básico verificando la salida generada.

Guía completa: [robfram-guide/Capitulo01/Practica1.md](#)



Práctica 2: Análisis del reporte HTML generado

Objetivo:

Interpretar output.xml, log.html y report.html de una ejecución; escribir un script Python que analice las métricas con pytest.

Guía completa: [robfram-guide/Capitulo01/Practica2.md](#)



Referencias Bibliográficas

Capítulo 1: Fundamentos de Automatización y Robot Framework

- ▶ <https://robotframework.org/>
- ▶ <https://www.ibm.com/topics/rpa>
- ▶ <https://glossary.istqb.org/>
- ▶ <https://www.selenium.dev/documentation/>
- ▶ <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html>